TIA — Enterprise Change Impact Intelligence

Product Requirements Document (PRD)

Product: TIA — Technical Impact Analysis
Version: 2.0
Target market: Banks and financial-services engineering organisations
Product category: Enterprise software change intelligence
Primary integration: GitHub Enterprise
Document status: Proposed product specification
Location: `/Users/aurthurj/Downloads/TIA`
Last updated: 2026-10-28

---

1. Executive Summary

TIA is an evidence-driven change-impact intelligence platform that determines how a proposed software change may affect an enterprise application landscape.

Modern banks operate interconnected systems spanning payment processing, customer onboarding, lending, fraud detection, reconciliation, reporting, APIs, event streams, databases, and deployment environments. These systems are often distributed across many repositories and teams.

Existing tools provide important capabilities: code search, code review, static analysis, dependency scanning, CI/CD, test execution, and runtime observability. TIA does not attempt to replace these tools.

Instead, TIA correlates their evidence into a unified, version-aware technical dependency graph and uses that graph to produce a **change-impact report**.

When an engineer changes a payment API, for example, TIA should identify known downstream consumers, affected schemas, related tests, deployed services, ownership information, and unresolved dependencies. It should explain why each component may be affected and identify which checks are needed before release.

The product must distinguish between:

- A dependency confirmed by source code or a contract.
- A dependency observed in runtime telemetry.
- A possible dependency inferred from incomplete evidence.
- A dependency that cannot currently be determined.

TIA must never present an unverified relationship as a confirmed fact.

Product vision

Make every consequential software change explainable across code, services, contracts, tests, deployments, and operational history.

Core value proposition

Before a change is merged or released, TIA provides an evidence-backed answer to five questions:

1. What could this change affect?
2. Why does TIA believe each component is affected?
3. Which tests and contract checks should be considered?
4. What is unknown or insufficiently verified?
5. What evidence and approvals are required by the organisation's release policy?

Core deliverable

The core deliverable is **not** a graph visualisation, an API inventory page, or an AI-generated chat answer.

The core deliverable is an **explainable Change Impact Report** (structured + human-readable) for a proposed revision, with evidence paths, unknowns, and policy outcomes.

---

2. Problem Statement

2.1 The enterprise engineering problem

A change in one repository can affect systems maintained by different teams, deployed independently, and documented in separate tools.

The relevant information may be spread across:

- GitHub repositories, branches, commits, and pull requests.
- Source code and function-level call relationships.
- REST APIs, asynchronous events, and message schemas.
- Database tables, views, stored procedures, and migrations.
- Unit, integration, contract, and end-to-end tests.
- CI/CD results, build artifacts, and deployment records.

- Runtime traces and service-to-service communication.
- Incidents, rollbacks, and previous changes.
- Service ownership, criticality, and release policies.

An engineer can inspect an individual repository or use a static-analysis tool, but determining the enterprise-wide impact often requires manually correlating information across multiple systems.

2.2 Example

A developer changes the `paymentStatus` field in a payment service.

The potential downstream impact includes:

- An API consumed by a mobile banking application.
- An event consumed by a reconciliation service.
- A fraud-monitoring consumer.
- A notification service.
- A reporting pipeline.
- A database mapping or transformation.
- Contract tests and integration tests.
- Services deployed in production that use the changed contract.

The challenge is not simply finding references to `paymentStatus`. It is determining which references belong to which systems, which versions are deployed, which relationships are supported by evidence, and which checks matter for this particular change.

2.3 Why another code graph is insufficient

A graph of files, classes, and functions is useful but is not, by itself, a compelling enterprise product.

Customers already have (or can adopt) code search, AI coding assistants, repository knowledge graphs, MCP tool wiring, static analysis, and observability. A product that only “maps code for chat” will be compared unfavourably to those stacks.

TIA must create value beyond a repository-level graph by correlating:

1. **Code evidence** — what the source appears to do.
2. **Contract evidence** — how systems communicate.
3. **Execution evidence** — what has been observed running.
4. **Test evidence** — what has been tested and with what result.
5. **Deployment evidence** — what versions are running and where.
6. **Operational evidence** — what has failed historically.
7. **Governance evidence** — which policies apply to the change.

The differentiator is the quality of the combined **change-impact report**, not the existence of the graph itself.

2.4 Lessons from prior product failure (binding constraints)

A prior document-centric “knowledge fabric” demo was rejected. Those failure modes are forbidden in TIA:

Failure	Binding rule
Answers only from documents	Impact authority is technical evidence (code, contracts, tests, runtime, deploy)
Invented API named from query text (“all api”)	Census and findings come from extracted/confirmed entities only
No function → chain → consumer path	Every material finding needs an evidence path
Force API catalog upload when none exists	Discover APIs/contracts from code and registries; catalogs are optional enrichers
Impression of dumping entire monorepos onto a permanent god-graph	Incremental, revision-aware, bounded surfaces; report-first UX
LLM used to invent missing dependencies	Deterministic traversal and evidence rules; LLM may only summarise established findings

3. Product Goals

3.1 Primary goals

G1. Cross-repository change-impact analysis

Determine potential downstream impact across repositories and services, not just within the changed file or repository.

G2. Evidence-backed explanations

Every material impact finding must include an explanation, supporting evidence, source reference, and confidence or verification status.

G3. Static and runtime correlation

Combine source-derived relationships with available runtime telemetry without treating either source as complete.

G4. Regression planning

Identify relevant existing tests and required verification areas using dependency relationships, contract changes, test results, and available coverage evidence.

G5. Version-aware operational context

Correlate changes with builds, deployments, environments, ownership, and historical incidents when the necessary data is available.

G6. Enterprise release-policy integration

Allow banks to apply configurable release checks based on service criticality, contract compatibility, mandatory tests, unresolved impact, and required approvals.

G7. Honest uncertainty

Explicitly identify missing repositories, incomplete telemetry, unresolved consumers, unavailable test evidence, and unsupported analysis.

3.2 Secondary goals

- Reduce manual investigation during change reviews.
- Improve consistency of impact assessments between teams.
- Make technical dependencies understandable to engineering leads and release managers.
- Preserve a traceable record of why a change was classified as low, medium, or high impact.
- Support restricted-network and enterprise deployment models.
- Allow coding assistants (via MCP or API) to **retrieve** Impact Reports — never to author confirmed dependencies.

3.3 Non-goals

TIA is not intended to:

- Replace GitHub code review or pull-request workflows.
- Replace CodeQL, security scanners, or dependency vulnerability scanners.
- Replace CI/CD or execute tests itself in the initial release.
- Replace AI coding assistants (Claude, Copilot, Cursor) as the developer UI.
- Replace general-purpose code knowledge graphs used for assistant context.
- Guarantee that every possible dependency will be discovered.
- Claim that a change is safe solely because no impact was found.
- Automatically approve or deploy high-risk banking changes.
- Use an LLM to invent missing dependencies or unsupported evidence.
- Build a universal business ontology before validating the technical product.
- Require an LLM for deterministic graph traversal or basic impact computation.
- Present a full-estate graph dump as the primary user experience.

---

4. Target Customers and Users

4.1 Initial customer segment

Banks and financial-services organisations with:

- Multiple repositories and independently deployed services.
- Distributed engineering ownership.
- Significant API and event-driven integration.
- Existing CI/CD and code-analysis tooling.
- Formal change-management or release-governance processes.
- A need to explain technical impact for consequential changes.

The first deployment should focus on one bounded application landscape rather than attempting to map an entire bank immediately.

4.2 Primary personas

Software engineer

Needs to understand which systems, APIs, and tests could be affected by a proposed change.

Senior engineer or technical lead

Needs to review cross-service impact, identify missing verification, and assess whether the proposed implementation is adequately tested.

Engineering manager

Needs to understand blast radius, ownership, and coordination requirements across teams.

Release manager

Needs evidence that the required checks and approvals have been completed before release.

Platform engineering team

Needs to integrate repository, CI/CD, deployment, and observability data into a consistent technical model.

Architecture or application owner

Needs a traceable view of dependencies between services, contracts, data stores, and deployed components.

5. Core User Journeys

5.1 Pull-request impact analysis

1. An engineer opens or updates a pull request.
2. TIA obtains the base and proposed-head revisions.
3. TIA identifies changed files and changed symbols where supported.
4. TIA determines which APIs, contracts, data entities, and downstream components may be affected.
5. TIA correlates the impact with available tests, runtime evidence, deployment information, and ownership.
6. TIA produces an impact report with evidence and unresolved questions.
7. TIA publishes a report or status check in the pull request.
8. The engineer or reviewer resolves missing evidence or completes the required checks.

5.2 API contract change

1. A developer changes an API definition or event schema.
2. TIA compares the base and proposed contract.
3. TIA identifies breaking or potentially breaking changes using configured compatibility rules.
4. TIA resolves known consumers through code, contract registries, or runtime evidence.
5. TIA identifies consumer-specific tests and verification requirements.
6. TIA reports affected consumers and any consumer relationships that remain unresolved.

5.3 Production incident investigation

1. An operator selects an incident, deployment, or failure signature.
2. TIA resolves the affected runtime component and deployed version when evidence exists.
3. TIA traces known relationships to relevant code, contracts, data stores, and changes.
4. TIA presents candidate changes and dependencies for investigation.
5. Each candidate is labelled according to its evidence.

TIA must not state that a change caused an incident solely because it occurred before the incident.

5.4 Release-readiness assessment

1. A release candidate is associated with a commit, build, and target environment.
2. TIA calculates the impact of the included changes.
3. It evaluates the configured release policy.

4. It checks the availability and results of required tests and contract checks.
5. It reports satisfied, failed, and unevaluated policy requirements.
6. The authorised release process decides whether to proceed.

---

6. Product Scope and Requirements

Priority definitions:

- **P0:** Required for the first usable release.
- **P1:** Required for a mature production pilot.
- **P2:** Later enhancement.

6.1 Repository and change ingestion

FR-001 — Repository connection (P0)

TIA shall connect to explicitly authorised GitHub repositories.

Requirements:

- Read repository metadata and source code.
- Identify default branches and relevant revisions.
- Retrieve pull-request metadata and changed files where authorised.
- Support private repositories without requiring public exposure.
- Record which repositories were successfully scanned.
- Report permission failures and inaccessible repositories.

FR-002 — Revision-aware analysis (P0)

TIA shall distinguish the base revision from the proposed revision.

A pull-request analysis must use the actual revision identifiers rather than silently analysing the current default branch.

The report must identify the commit or revision used for each analysis.

FR-003 — Incremental analysis (P0)

TIA should reanalyse changed components and the relationships potentially affected by those changes instead of rescanning every repository for every pull request.

A full rescan must remain available for recovery, parser upgrades, and graph reconciliation.

FR-004 — Supported languages (P0/P1)

Initial implementation:

- Python: P0.
- Java: P1, unless validated customer demand requires Java in the first pilot.

Pilot note: Many bank core services are Java. If the pilot landscape is Spring/Java-dominant, promote Java to P0 for that pilot rather than forcing a Python-only story.

The architecture must support adding additional languages through separate language-analysis adapters.

The initial product must clearly report language and framework coverage. Unsupported code must not be represented as fully analysed.

FR-005 — Repository freshness (P0)

TIA shall track:

- Last successful scan.
- Source revision analysed.
- Parser or extractor version.
- Scan failures.
- Stale relationships.
- Missing or inaccessible repositories.

A partial scan must not be presented as a complete scan.

6.2 Technical dependency graph

FR-006 — Core graph entities (P0)

The initial graph shall support the following entities:

- Repository.
- Commit or revision.
- File.
- Function or method.
- API endpoint.
- Service.
- Test case.
- Unit or integration test.
- Data entity.
- Database table or view.
- API or event schema.
- Service owner, where available.

FR-007 — Execution and operations entities (P1)

The graph shall support:

- Build artifact.
- Deployment.
- Environment.
- Runtime trace or observed dependency.
- Incident.
- Failure signature.
- Rollback.
- Service criticality.
- Release policy.

FR-008 — Explicit relationship types (P0)

Relationships shall be typed rather than represented as generic links.

Examples:

- `CALLS`
- `EXPOSES_ENDPOINT`
- `CONSUMES_API`
- `PRODUCES_EVENT`
- `CONSUMES_EVENT`
- `USES_SCHEMA`
- `READS_TABLE`
- `WRITES_TABLE`
- `TESTS_SYMBOL`
- `BUILT_FROM`
- `DEPLOYED_AS`
- `OBSERVED_CALL_TO`
- `OWNED_BY`
- `ASSOCIATED_WITH_INCIDENT`

The implementation may use different internal names, but each relationship must have a defined meaning.

FR-009 — Relationship provenance (P0)

Every material relationship must preserve:

- Source system.
- Source file, symbol, or record.
- Revision or time range, where applicable.
- Extraction method.
- Evidence reference.
- Verification status.

- Confidence or certainty classification.
- Last-seen or validity information.

FR-010 — Evidence classifications (P0)

TIA shall distinguish at least these states:

Classification	Meaning
Source-confirmed	Supported by an explicit source-code or configuration relationship
Contract-confirmed	Supported by an API, event, or schema definition
Runtime-observed	Observed in available runtime telemetry
Corroborated	Supported by multiple independent evidence sources
Inferred	Derived from indirect evidence and requiring validation
Unknown	Insufficient evidence to determine the relationship
Stale	Evidence may no longer describe the current revision or deployment

A source-confirmed relationship is not necessarily proof that the path executes in production. A runtime-observed relationship is evidence of observed execution, not proof of all possible execution paths.

Trust rule: Only classifications allowed by the active analysis/policy configuration may appear as confirmed impact. Inferred relationships must be labelled and must not be silently upgraded.

FR-011 — Temporal relationships (P1)

TIA shall preserve historical versions of material relationships when their meaning depends on a commit, build, deployment, or environment.

The graph must be able to answer questions such as:

- Which consumers were known to use this contract at the time of a release?
- Which version of a service was deployed when an incident occurred?
- Which dependencies changed between two releases?

6.3 Cross-repository and cross-service impact

FR-012 — Changed-symbol identification (P0)

TIA shall identify changed files and, where supported, changed functions, methods, classes, and configuration elements.

If symbol-level comparison is unavailable, the report shall state that the analysis is file-level rather than silently claiming symbol-level precision.

FR-013 — Dependency traversal (P0)

TIA shall traverse the relevant graph from changed components to potential downstream dependants.

The traversal must support both direct and transitive dependencies.

For example:

`Changed Function → API Endpoint → Consumer Service → Consumer Test`

The report must preserve the path connecting each affected component to the original change.

FR-014 — Cross-repository resolution (P0)

TIA shall connect relationships across repository boundaries when supported by resolvable evidence, including:

- API specifications.
- Event schemas.
- Explicit service configuration.
- Importable or generated clients.
- Shared data contracts.
- Runtime service relationships.

Matching names alone must not be sufficient to claim a confirmed dependency.

FR-015 — Impact scope (P0)

TIA shall classify findings into:

- Direct impact.
- Transitive impact.
- Contract impact.
- Data impact.
- Test impact.
- Runtime or deployment impact.
- Operational or historical association.
- Unresolved potential impact.

FR-016 — Bounded traversal (P0)

The system shall enforce configurable traversal depth, result limits, and analysis time budgets.

When a limit is reached, the report must indicate that the result is truncated or incomplete.

TIA must not silently omit potentially relevant findings and label the analysis complete.

6.4 API and event contract intelligence

FR-017 — API discovery (P0)

TIA shall discover supported API definitions from source code and configuration, including framework-supported route declarations.

For each endpoint, record where available:

- HTTP method.
- Path.
- Service and repository.
- Request and response schema.
- Authentication or authorisation configuration.
- Source revision.
- Related tests.

No catalog gate: Absence of an uploaded OpenAPI/Swagger document must not prevent discovery of endpoints declared in supported frameworks. External catalogs, if connected, enrich and corroborate; they do not define the only path to an API inventory.

FR-018 — API contract comparison (P1)

TIA shall compare base and proposed API definitions and identify changes such as:

- Removed endpoint.
- Changed HTTP method or path.
- Removed field.
- Field type change.
- Requiredness change.
- Enum narrowing.
- Response shape change.
- Authentication or authorisation configuration change.

Compatibility classification must follow configurable rules appropriate to the protocol and schema format.

FR-019 — Event and message contracts (P1)

TIA shall support configured event or message schemas, including applicable formats such as JSON Schema and Avro.

It shall identify known producers, consumers, and schema dependencies when evidence is available.

FR-020 — Consumer impact report (P1)

For each changed contract, TIA shall report:

- Changed contract elements.
- Known affected consumers.
- Evidence for each consumer relationship.
- Compatibility classification.
- Relevant tests.
- Unresolved consumers or missing contract information.

TIA must not claim that a contract change is backward compatible without evaluating the applicable compatibility rules.

6.5 Test intelligence

FR-021 — Static test association (P0)

TIA shall link tests to functions, endpoints, contracts, or services when a defensible relationship can be established from imports, test structure, naming conventions, explicit configuration, or other supported evidence.

The association method must be recorded.

FR-022 — CI test results (P1)

TIA shall ingest authorised CI results, including:

- Test identity.
- Test result.
- Execution time.
- Commit or build identifier.
- Environment, where available.
- Test report or evidence reference.

FR-023 — Coverage enrichment (P1)

Where available, TIA shall ingest coverage or execution evidence linking tests to source symbols.

TIA must distinguish:

- Test exists.
- Test is statically associated with a component.
- Test passed in a particular run.
- Test executed a particular component.
- Test validated a particular contract.

These states are not interchangeable.

FR-024 — Regression recommendations (P1)

TIA shall recommend tests based on affected components and available test relationships.

Recommendations must explain why a test is relevant.

TIA must not claim that unselected tests can safely be skipped solely because no graph path was found.

FR-025 — Missing-test evidence (P1)

TIA shall identify potentially important impact areas for which no relevant test evidence was found.

The report must say “no relevant test evidence found in the analysed sources,” not “no tests exist,” unless the available test inventory is known to be complete.

6.6 Runtime and deployment intelligence

FR-026 — Runtime integration (P1)

TIA shall support ingestion of authorised runtime dependency evidence from supported observability sources, such as OpenTelemetry-compatible traces.

The initial implementation may use imported or queried telemetry rather than requiring TIA to collect traces directly.

FR-027 — Runtime relationship classification (P1)

TIA shall label runtime relationships with the observed service, environment, time window, and available deployment or version identifiers.

Absence of a runtime observation must not be interpreted as proof that a dependency does not exist.

FR-028 — Deployment correlation (P1)

TIA shall link services to build artifacts and deployments where reliable identifiers are available.

The system must distinguish the proposed code revision from the revision currently deployed in each environment.

FR-029 — Environment-specific impact (P1)

TIA shall support queries that distinguish development, testing, staging, and production environments.

If environment mapping is incomplete, the report must explicitly identify the limitation.

6.7 Historical change and incident intelligence

FR-030 — Change history (P1)

TIA shall retain authorised historical information about changes and relevant graph relationships.

It should support comparisons between releases and identification of dependencies introduced, removed, or modified over time.

FR-031 — Incident association (P1)

TIA shall support linking incidents to affected services, deployments, time windows, and related changes when evidence is available.

Each association must identify whether it is:

- Directly recorded.
- Time-correlated.
- Dependency-derived.
- Manually validated.

FR-032 — Historical investigation (P1)

TIA shall allow users to investigate which changes and components are relevant to a historical incident.

It must not automatically classify a historically associated change as the root cause.

6.8 Release-policy integration

FR-033 — Configurable policy rules (P1)

Authorised administrators shall be able to configure release requirements based on criteria such as:

- Service criticality.
- Breaking API or event changes.
- Mandatory test results.
- Required contract checks.

- Missing owner approval.
- Unresolved high-consequence dependencies.
- Stale or incomplete impact analysis.
- Required evidence for production deployment.

FR-034 — Policy evaluation (P1)

Each policy rule shall produce one of the following outcomes:

- Pass.
- Fail.
- Not evaluated.
- Requires review.

A missing data source must not automatically produce a pass.

FR-035 — GitHub integration (P1)

TIA shall publish its findings to the pull request through a supported integration, such as a check result or comment.

The integration shall include:

- Impact summary.
- Critical findings.
- Required verification.
- Report link or identifier.
- Analysis revision.
- Incomplete-analysis warning, when applicable.

FR-036 — Advisory and enforcement modes (P1)

TIA shall support:

1. Advisory mode — report findings without blocking merges.
2. Policy mode — report policy outcomes to the configured CI/CD or repository protection process.

The customer must explicitly configure enforcement. TIA shall not independently deploy code or override repository protection settings.

FR-037 — Assistant / MCP distribution (P1)

TIA may expose read APIs or an MCP server so coding assistants can fetch an existing Impact Report by analysis id / PR id.

Requirements:

- Tools return structured report JSON.

- Assistants may narrate only fields present in the report.
- Assistants must not be able to write confirmed relationships through MCP in the initial design.
- Org guidance: “no blast-radius claim in review without a TIA analysis id.”

---

7. Impact Analysis Algorithm

The impact engine is the central technical capability of TIA.

7.1 Inputs

The engine receives:

- Base revision.
- Proposed revision.
- Changed files and symbols.
- Relevant graph subgraph.
- Contract differences.
- Available test evidence.
- Runtime and deployment evidence.
- Applicable policy rules.
- Analysis limits and configuration.

7.2 Processing stages

Stage 1 — Change detection

Identify changed components and classify the change type:

- Implementation change.
- API change.
- Event-schema change.
- Database-schema change.
- Configuration change.
- Test-only change.
- Dependency or build change.

Stage 2 — Graph resolution

Resolve changed components to known graph entities and determine which relationships can be traversed.

Unresolved symbols, ambiguous service identities, and inaccessible repositories must be recorded.

Stage 3 — Impact traversal

Traverse relevant dependency relationships to identify direct and transitive impact.

Traversal rules shall be relationship-aware. For example, a write dependency may have different implications from a test association or an incident association.

Stage 4 — Contract and data analysis

Compare changed contracts or schemas against their previous versions and identify known consumers or data dependencies.

Stage 5 — Test and runtime correlation

Associate affected components with relevant tests, prior CI outcomes, coverage evidence, observed runtime relationships, and deployed versions.

Stage 6 — Evidence assessment

For each finding, record the evidence path, verification state, source freshness, and unresolved limitations.

Stage 7 — Policy evaluation

Evaluate applicable release rules using the available evidence.

Stage 8 — Report generation

Produce a structured report and a human-readable summary.

7.3 Impact severity

TIA shall use configurable severity categories:

Severity	Definition
Critical	A change may affect a critical system or violate a mandatory compatibility or release policy
High	A consequential downstream dependency or important contract is affected
Medium	A relevant downstream component or verification requirement is identified
Low	Limited, localised impact is supported by the available evidence
Unknown	The evidence is insufficient to assess the impact reliably

Severity must be based on documented rules and available evidence. It must not be presented as a statistically calibrated probability unless a validated model supports that claim.

---

8. Change Impact Report

Every analysis shall produce a structured, machine-readable report and a readable summary.

8.1 Required report fields

- Analysis identifier.
- Repository and pull-request identifiers, where applicable.
- Base and proposed revisions.
- Scan timestamp.
- Analysis status.
- Changed components.
- Affected components.
- Severity and rationale.
- Evidence paths.
- Relevant tests and their evidence status.
- Contract findings.
- Runtime and deployment findings.
- Unresolved dependencies.
- Missing data sources.
- Truncation or time-budget warnings.
- Policy results.
- Recommended next checks.

8.2 Example output

The following is illustrative output, not a claim about a real repository.

```
{
  "analysis_id": "tia-pr-1842",
  "status": "complete_with_gaps",
  "change": {
    "repository": "payments-service",
    "base_revision": "base-sha",
    "proposed_revision": "head-sha",
    "changed_components": [
      {
        "type": "api_contract",
        "name": "PaymentStatus"
      }
    ]
  },
  "findings": [
    {
      "component": "reconciliation-service",
      "impact_type": "contract",
      "severity": "high",
      "evidence_status": "contract-confirmed",
      "reason": "The consumer references the changed PaymentStatus schema.",
      "required_verification": [
        "Run the reconciliation contract tests",
        "Verify compatibility with the deployed consumer version"
      ]
    },
    {
      "component": "fraud-monitoring-service",
      "impact_type": "runtime",
      "severity": "medium",
      "evidence_status": "runtime-observed",
      "reason": "Telemetry recorded communication between the services during the analysed period."
      "limitations": [
        "Runtime evidence does not establish complete consumer coverage"
      ]
    }
  ],
  "unknowns": [
    {
      "area": "external_consumers",
      "reason": "The external consumer inventory was not available."
    }
  ],
  "policy_results": [
    {
      "rule": "required_contract_tests",
      "result": "not_evaluated",
      "reason": "The CI test result was unavailable."
    }
  ]
}
```

8.3 Report integrity

The report must distinguish a successful analysis with no identified impacts from:

- A scan that did not run.
- A failed scan.
- A partial scan.
- An analysis performed against stale data.
- A result truncated by resource limits.
- An analysis that could not resolve the changed component.

A clean result is meaningful only when the scope and limitations of the analysis are visible.

8.4 Anti-hallucination integrity

The following behaviours are defects, not UX quirks:

- Creating an entity whose name is derived from query phrasing (e.g. an API named “all api”).
- Promoting an inferred relationship to source-confirmed without new evidence.
- Omitting the unknowns panel when consumer inventory or scan coverage is incomplete.
- Claiming “safe to skip tests” solely because no path was found.
- Treating “no runtime observation” as “no dependency.”

---

9. User Interface Requirements

9.1 Pull-request impact view

The primary interface shall show:

1. Overall impact classification.
2. Changed components.
3. Most consequential affected services.
4. Contract and data changes.
5. Relevant tests and their results.
6. Policy outcomes.
7. Unknowns and missing evidence.

Users must be able to navigate from a finding to its supporting source.

9.2 Impact path explorer

Users shall be able to inspect a dependency path such as:

`Changed API → Event Producer → Event Schema → Reconciliation Consumer → Contract Test`

The interface must identify the evidence supporting each edge and distinguish confirmed relationships from inferred relationships.

A large graph visualisation must not be the only way to understand the result. The default experience should prioritise the impact report, with graph exploration available as supporting functionality.

9.3 Filters

The report shall support filtering by:

- Severity.
- Service or repository.
- Direct versus transitive impact.
- Contract, data, test, runtime, and operational impact.
- Evidence status.
- Environment.
- Policy result.
- Unresolved or stale evidence.

9.4 Unknowns panel

The UI shall display missing information that materially affects confidence in the assessment.

Examples:

- Consumer repository not connected.
- Contract registry unavailable.
- No relevant coverage data found.
- Runtime telemetry does not cover the target period.
- Deployment version could not be resolved.
- Analysis reached its configured traversal limit.

---

10. System Architecture

10.1 Logical architecture

Layer A — Source integrations

Connectors for:

- GitHub Enterprise.

- Source code and configuration.
- API and event-schema registries.
- CI/CD test reports.
- Coverage reports.
- OpenTelemetry-compatible telemetry.
- Deployment systems.
- Incident-management systems.
- Service ownership and policy registries.

Each connector shall operate with explicit permissions and report its coverage.

Layer B — Evidence extraction

Language and source adapters extract:

- Symbols and calls.
- API endpoints.
- Contract definitions.
- Data access relationships.
- Test associations.
- Build and deployment metadata.

The output shall retain provenance back to the original source.

Optional adapters may import graphs produced by other AST/knowledge-graph tools when provenance can be preserved. Import does not transfer authority: TIA still produces the Impact Report under TIA evidence rules.

Layer C — Identity and relationship resolution

Resolve identities across repositories and systems.

Examples include linking an API declaration to a consumer configuration or a service name in telemetry to a repository and deployment.

Ambiguous matches must remain unresolved or inferred until corroborated.

Layer D — Technical dependency graph

Store entities, typed relationships, evidence, and temporal validity.

The graph shall support bounded traversals, revision-aware analysis, and targeted retrieval.

Layer E — Change-impact engine

Compare revisions, traverse affected dependencies, assess contracts, associate tests, correlate runtime evidence, and evaluate policies.

Layer F — Report and integrations

Produce structured reports, a user interface, GitHub pull-request results, and optional assistant/MCP read access to reports.

10.2 Architecture principle

The graph is the evidence and relationship layer. The impact engine determines what to do with that evidence. The report communicates conclusions and limitations.

Do not confuse graph construction with impact analysis.

10.3 Storage

The implementation should use storage selected for the first deployment's scale and operational requirements.

For an early local prototype, SQLite can be sufficient for scan metadata and a bounded graph.

For a multi-repository enterprise deployment, the design must evaluate:

- Concurrent access.
- Incremental updates.
- Historical revisions.
- Traversal performance.
- Auditability.
- Backup and recovery.
- Retention.
- Operational support.

The PRD does not mandate a particular graph database. Storage selection must be validated against the expected workload.

10.4 LLM usage

The first version must not depend on an LLM to determine graph reachability or invent dependency relationships.

An LLM may optionally summarise already-established findings, but:

- Structured findings must be generated by deterministic logic.
- Each material claim must refer to evidence.
- Unsupported claims must be excluded.
- Summaries must preserve unknowns and qualifications.
- The report must remain useful if an LLM is unavailable.

11. Security, Privacy, and Enterprise Deployment

Security is a product requirement, not a later enhancement.

11.1 Access control

- Use least-privilege access to repositories and integrations.
- Do not request write access when read access is sufficient.
- Separate read permissions from any optional ability to publish pull-request checks.
- Respect repository and organisation access boundaries.
- Do not expose information from repositories the requesting user is not authorised to access.

11.2 Deployment models

The architecture should support:

- Local developer use for early evaluation.
- Private enterprise deployment.
- Restricted-network or on-premises deployment where required by the customer.

The initial pilot must specify exactly where source code, graph data, reports, and telemetry are processed and stored.

11.3 Audit

Record:

- Repository access and scan events.
- Source revisions analysed.
- Connector failures.
- Analysis requests.
- Report generation.
- Policy evaluations.
- Administrative configuration changes.
- Evidence updates and invalidations.

11.4 Data retention

Customers must be able to configure retention for:

- Source-derived graph records.
- Historical analysis reports.
- Runtime metadata.
- Incident associations.
- Audit records.

Secrets, credentials, and unnecessary personal data must not be copied into graph records or reports.

11.5 Integrity

TIA shall preserve the source and revision behind each finding so that an auditor or engineer can reproduce or investigate the result.

Changes to extraction logic must not silently rewrite historical reports without identifying the new analysis version.

---

12. Performance and Reliability Requirements

The following are initial engineering targets to validate in the pilot, not established performance claims.

Requirement	Initial target
Incremental PR analysis	P95 within 2 minutes for a defined small-repository fixture
Interactive impact report	P95 within 5 seconds after the analysis is complete
Analysis determinism	Same inputs, extractor version, configuration, and graph snapshot produce the same structured findings
Resource limits	Configurable traversal, memory, and time limits
Failure reporting	Connector and analysis failures are visible, not silently ignored
Incremental consistency	Changes invalidate or refresh affected relationships
Reproducibility	Every report records the revisions and analysis configuration used
Availability	Set according to the deployment tier and pilot requirements

Performance benchmarks must use representative repositories and documented test fixtures.

---

13. MVP Scope

The MVP should prove the central product hypothesis: cross-repository impact intelligence provides useful information beyond an individual repository's code analysis.

MVP Phase 1 — Code and contract impact

Priority: P0

Deliver:

- GitHub repository ingestion.
- Base-versus-head change detection.
- Python source analysis (Java if pilot requires).
- File and function-level graph.
- API endpoint discovery from code.
- Explicit graph relationships and provenance.
- Cross-repository API consumer resolution where evidence exists.
- Bounded transitive impact analysis.
- Basic test association.
- Structured JSON Change Impact Report.
- CLI and basic human-readable report.
- Clear handling of partial and failed scans.

Do not require runtime telemetry, incident correlation, or automated release enforcement to demonstrate the first value.

Phase 1 demo rule: Use a bounded 2+ repository fixture with a real contract consumer path. Do not demo open-ended chat against documents.

Phase 2 — Enterprise evidence enrichment

Priority: P1

Deliver:

- CI test results.
- Coverage integration.
- API and event contract comparison.
- Runtime dependency evidence.
- Deployment-version correlation.

- Service ownership and criticality.
- GitHub pull-request reporting.
- Configurable release-policy evaluation.
- Optional MCP/API report retrieval for assistants.

Phase 3 — Operational change intelligence

Priority: P1/P2

Deliver:

- Historical graph comparisons.
- Incident and rollback association.
- Release-to-release impact comparisons.
- Environment-specific dependency views.
- Improved service identity resolution.
- Policy administration and audit reporting.

Phase 4 — Advanced intelligence

Priority: P2

Evaluate after customer validation:

- Additional languages and frameworks.
- Broader event-stream and data-lineage analysis.
- Cross-application change-risk trends.
- Historical impact calibration using validated outcomes.
- Multi-team change coordination.
- Additional repository, observability, and deployment platforms.

---

14. Acceptance Criteria

The MVP is not complete merely because it can build a graph or list API endpoints.

AC-001 — Changed revision

Given a pull request with a known base and proposed revision, TIA reports the exact revisions analysed.

AC-002 — Direct impact

Given a changed function with a known caller, TIA identifies the caller and provides a source-backed path.

AC-003 — Transitive impact

Given a chain of supported dependencies across two or more repositories, TIA identifies the reachable downstream components within the configured analysis scope.

AC-004 — Contract impact

Given a changed API contract and a known consumer, TIA identifies the consumer and explains the relationship using contract or source evidence.

AC-005 — Unresolved consumer

Given a changed contract and an incomplete consumer inventory, TIA reports the known consumers and explicitly states that the inventory may be incomplete.

AC-006 — Test association

Given a test with an established relationship to an affected component, TIA includes the test and explains the association method.

AC-007 — Runtime distinction

Given a runtime-observed dependency, TIA labels it as observed and identifies the relevant time window and environment when available.

AC-008 — No false coverage claim

Given no runtime evidence for a dependency, TIA does not conclude that the dependency cannot occur.

AC-009 — Failed scan

Given an inaccessible repository or failed parser, TIA reports the failure and marks any resulting analysis as incomplete.

AC-010 — Zero-impact result

Given a successful analysis with no identified downstream impact, TIA returns a successful, scoped result with scan coverage and limitations. It does not confuse this result with a failed or empty scan.

AC-011 — Stale relationships

Given a relationship invalidated by a source change, TIA does not continue presenting it as current without identifying its stale status.

AC-012 — Bounded traversal

Given a graph that exceeds configured analysis limits, TIA reports truncation and does not claim complete impact discovery.

AC-013 — Policy failure

Given a mandatory contract check that failed, TIA reports the failed requirement and the supporting result.

AC-014 — Policy not evaluated

Given an unavailable CI result, TIA marks the relevant policy as not evaluated rather than passed.

AC-015 — Reproducible evidence

Given a report, an authorised reviewer can identify the source revisions and evidence used to produce its material findings.

AC-016 — Access control

Given a user without permission to a repository, TIA does not disclose that repository’s protected source or findings.

AC-017 — No invented entities

Given a request to list APIs or consumers when none were extracted, TIA returns an empty confirmed set with an explicit incompleteness or empty-extract status. It never invents entity names from the request phrasing.

---

15. Success Metrics

The pilot must measure whether TIA improves engineering decisions, not just whether it extracts graph entities.

15.1 Primary product metrics

Impact discovery

- Percentage of independently identified important dependencies discovered by TIA.
- Number of material dependencies found by TIA that the existing workflow missed.
- Rate of materially incorrect impact findings.

Engineering efficiency

- Time required to complete an impact assessment.
- Time spent manually locating downstream consumers.
- Time required to identify the relevant regression tests.
- Reviewer time spent investigating missing or ambiguous evidence.

Evidence quality

- Percentage of material findings with traceable evidence.
- Percentage of findings with resolvable source references.
- Rate of stale relationships.
- Percentage of analyses correctly identifying incomplete input coverage.

Operational usefulness

- Percentage of recommendations considered useful by engineers.
- Percentage of recommended checks that engineers confirm are relevant.
- Number of cross-team dependencies identified before release.
- Policy evaluation accuracy against independently reviewed cases.

15.2 Pilot targets

The following are proposed targets for a controlled pilot and must be validated against a baseline.

- At least 90% recall for a curated set of known, in-scope critical dependencies.
- At least 90% of material impact findings have traceable evidence.
- At least 30% reduction in median time to complete the selected impact-assessment workflow.
- No silent pass for missing mandatory evidence.
- No material finding presented as source-confirmed without supporting evidence.

Precision and recall must be measured against an independently reviewed dependency set. Results should be segmented by language, relationship type, and evidence source.

---

16. Risks and Mitigations

Risk 1 — The product duplicates existing tools

Mitigation: Integrate with existing code analysis, repository, CI/CD, and observability tools. Focus on cross-system impact correlation and evidence-backed decisions. Compete on the Impact Report + unknowns + policy — not on “we also have a code graph.”

Risk 2 — Cross-repository relationships are incomplete

Mitigation: Track repository coverage, use contract and runtime evidence, report unresolved consumers, and validate against curated enterprise dependency sets.

Risk 3 — Engineers distrust impact results

Mitigation: Show the evidence path for every material finding, expose extraction methods, and separate confirmed, observed, inferred, stale, and unknown relationships.

Risk 4 — The graph becomes too large

Mitigation: Use incremental ingestion, targeted graph expansion, bounded traversal, and query-specific reports rather than exposing the entire graph for every request.

Risk 5 — Runtime telemetry creates false confidence

Mitigation: Record the observation window and environment. Treat missing observations as unknown, not proof of absence.

Risk 6 — Historical correlation is mistaken for causation

Mitigation: Label incident relationships by evidence type and require independent validation before claiming root cause.

Risk 7 — The platform creates a security or compliance concern

Mitigation: Support least privilege, private deployment, audit logging, data minimisation, configurable retention, and customer-controlled enforcement.

Risk 8 — The scope becomes too broad

Mitigation: Begin with one language, one repository provider, a bounded set of contracts, and one measurable change-impact workflow. Pass decision gates before Phase 2/3 expansion.

Risk 9 — Bank pilot needs Java but MVP ships Python-only

Mitigation: Confirm pilot landscape early; promote Java to P0 when the bounded landscape is Spring-dominant.

---

17. Competitive Positioning

TIA should coexist with existing developer and security tools.

Capability	Existing engineering tools	TIA's intended role
Code search and navigation	Locate symbols and references	Correlate findings across repositories and systems
AI-assisted coding / review (Claude, Copilot, Cursor)	Review proposed code and suggest improvements	Supply evidence-backed Impact Reports those tools can cite, not invent
Repository knowledge graphs / assistant MCP maps	Map code for assistant context	Consume or coexist; own revision-aware cross-system impact + report integrity
Static analysis	Detect supported code defects and patterns	Use relevant analysis results as evidence for impact
Dependency and security scanning	Identify package dependencies and known risks	Connect relevant dependency changes to affected applications and releases

Capability	Existing engineering tools	TIA's intended role
CI/CD	Build, test, and deploy software	Interpret test and deployment evidence in the context of change impact; evaluate configured policies
Runtime observability	Show observed behaviour and service interactions	Correlate observed relationships with code, contracts, and proposed changes
TIA	—	Produce an explainable, cross-system Change Impact Report and evaluate configured verification policies

Intended differentiation (one paragraph for buyers)

Banks already have search, scanners, CI, observability, and increasingly AI assistants with code graphs. TIA's job is to answer, for a **specific proposed revision**, what else may be affected **across services**, **why** (evidence class + path), what is **unknown**, and whether **release policy** is satisfied — without pretending a chat answer or a repo map is a release decision.

This is an intended differentiation, not a claim that existing platforms cannot provide any overlapping functionality. The product must demonstrate incremental value in a customer environment.

---

18. Go-to-Market Validation

Before investing in a full enterprise platform, validate the product with one bank or financial-services engineering team.

Pilot scenario

Select a service with:

- At least two downstream consumers.
- A documented API or event contract.
- Multiple repositories.
- Existing automated tests.
- An established release workflow.

Perform a controlled change-impact assessment on a real or representative change.

Compare TIA with the customer's current process.

Measure:

1. Which important dependencies the existing workflow identified.
2. Which dependencies TIA identified that the workflow missed.
3. Which TIA findings were incorrect or irrelevant.
4. Whether engineers could verify the evidence.
5. Whether TIA reduced assessment time.
6. Whether the report identified missing test or contract evidence before release.

A successful pilot should demonstrate repeatable improvements in dependency discovery and assessment time without unacceptable false positives or unsupported claims.

---

19. Product Roadmap Decision Gates

Gate 1 — Technical feasibility

Can TIA reliably identify changed symbols and build source-backed relationships for the selected language and repositories?

Gate 2 — Cross-repository value

Can TIA discover important dependencies that engineers did not identify using their existing tools and workflow?

Gate 3 — Evidence trust

Can engineers understand, verify, and challenge each material finding?

Gate 4 — Workflow adoption

Will engineers use the report during pull-request review or release assessment without excessive noise?

Gate 5 — Enterprise readiness

Can the platform operate within the customer’s access-control, deployment, audit, retention, and performance requirements?

Do not expand into broad incident intelligence, sophisticated risk scoring, or automated release enforcement until the earlier gates demonstrate value.

---

20. Final Product Definition

TIA is an **enterprise change-impact intelligence platform** that connects code, APIs, event schemas, data dependencies, tests, runtime observations, deployments, and operational history into a version-aware evidence graph.

Its core deliverable is not a graph visualisation, an API inventory, or an AI-generated explanation.

Its core deliverable is an **explainable Change Impact Report** that identifies potentially affected systems, provides the evidence behind each finding, recommends relevant verification, reports unresolved dependencies, and evaluates configured release policies.

The initial product should concentrate on one measurable workflow: determining the cross-repository impact of a proposed API or service change.

The long-term opportunity is to become a trusted intelligence layer between software changes and enterprise release decisions—working with existing engineering tools rather than attempting to replace them.

Product success criterion: A bank's engineering team can use TIA to identify important cross-system dependencies, verify why they matter, and make a better-informed change or release decision than it could using its existing workflow alone.

---

21. Document history

Version	Date	Notes
0.1	2026-10-28	Basic code-scan / ask CLI PRD
0.2	2026-10-28	Competitive rewrite (authority vs assistant graphs)
2.0	2026-10-28	Enterprise Change Impact Intelligence PRD for banks; GitHub-first; report-centric; evidence classifications; phased MVP and gates. Minor additions: binding demo-failure constraints, AC-017, FR-037 MCP read path, Java pilot note, assistant coexistence

---

End of PRD v2.0